# 16.09 — T-SQL: переменные, выражения и условная логика

## Конспект

Локальная переменная начинается с @. DECLARE задаёт имя и тип, SET присваивает значение, SELECT может записывать результат запроса в переменные.

INT хранит целые числа, DECIMAL(p,s) - точные числа, NVARCHAR - Unicode-текст.
Арифметические операторы: +, -, *, /, %. Деление двух INT даёт целый результат.

Операторы сравнения формируют условия. AND требует выполнения обоих условий, OR - хотя бы одного, NOT отрицает условие.

IF ... ELSE выбирает выполняемые команды. BEGIN ... END объединяет команды в блок. CASE возвращает значение: простой CASE сравнивает выражение, поисковый проверяет отдельные условия WHEN сверху вниз.

NULL обозначает отсутствие значения. Его проверяют через IS NULL и IS NOT NULL. CAST и CONVERT преобразуют типы. CASE удобно использовать для возрастной группы и результата обучения.

## Ответы на вопросы для самопроверки

1. Что такое переменная в TSQL?
Переменная - именованное место для хранения значения заданного типа.

2. С какого символа начинаются локальные переменные?
С символа @.

3. Для чего используется DECLARE?
DECLARE объявляет переменную.

4. Для чего используется SET?
SET присваивает переменной значение.

5. Чем SET отличается от SELECT при работе с переменными?
SET присваивает значение одной переменной. SELECT может присвоить значения нескольким переменным из запроса; если строк нет, прежнее значение сохраняется.

6. Для чего используется тип INT?
INT хранит целые числа.

7. Для чего используется DECIMAL?
DECIMAL хранит точные числа с заданным числом цифр и знаков после запятой.

8. Чем VARCHAR отличается от NVARCHAR?
NVARCHAR хранит Unicode; VARCHAR хранит текст с учётом кодировки и collation.

9. Для чего используется оператор %?
% возвращает остаток от деления.

10. Что произойдёт при выполнении 10 / 3 , если оба значения имеют тип INT?
Результат 10 / 3 при двух INT равен 3.

11. Для чего используются операторы = , <> , > , < , >= , <=?
Они сравнивают значения: равно, не равно, больше, меньше, больше или равно, меньше или равно.

12. Для чего используется AND?
AND объединяет условия, которые должны выполняться одновременно.

13. Для чего используется OR?
OR требует выполнения хотя бы одного условия.

14. Для чего используется NOT?
NOT отрицает условие.

15. Для чего нужен IF ... ELSE?
IF ... ELSE выбирает, какой набор инструкций выполнить.

16. Зачем используются BEGIN ... END?
BEGIN ... END объединяет инструкции в блок.

17. Что такое CASE?
CASE - выражение, которое выбирает возвращаемое значение по условию.

18. Чем CASE отличается от IF?
CASE возвращает значение, IF управляет выполнением инструкций.

19. В каком порядке проверяются условия WHEN?
WHEN проверяются сверху вниз; используется первая подходящая ветка.

20. Чем простой CASE отличается от поискового?
Простой CASE сравнивает одно выражение с вариантами. Поисковый CASE проверяет самостоятельные логические условия.

## Практическая часть

## Подготовка таблицы после задания 10.09

Используется таблица и пять студентов из 10.09. Подготовку выполнить один раз; если столбец average_grade уже есть, ALTER TABLE пропустить.

```
USE CollegeDB;
GO
ALTER TABLE college.students ADD average_grade DECIMAL(4,2);

UPDATE college.students
SET average_grade = CASE id
    WHEN 1 THEN 4.7
    WHEN 2 THEN 4.2
    WHEN 3 THEN 3.6
    WHEN 4 THEN 4.9
    WHEN 5 THEN 2.8
END
WHERE id BETWEEN 1 AND 5;
GO
```

## Задания 1-3

Результаты на учебных данных: 5, Иван, Совершеннолетний.

```
DECLARE @studentCount INT;
SELECT @studentCount = COUNT(*) FROM college.students;
SELECT @studentCount AS student_count;
GO

DECLARE @studentName NVARCHAR(100);
SELECT @studentName = name FROM college.students WHERE id = 1;
SELECT @studentName AS student_name;
GO

DECLARE @age INT;
SELECT @age = age FROM college.students WHERE id = 1;
IF @age >= 18
    PRINT N'Совершеннолетний';
ELSE
    PRINT N'Несовершеннолетний';
GO
```

## Задания 4-5

Иван и Анна - отличники, Мария и Алексей - хорошисты, Дмитрий имеет задолженности.

```
SELECT *,
    CASE WHEN age >= 18 THEN N'Совершеннолетний'
         ELSE N'Несовершеннолетний' END AS age_status
FROM college.students;

SELECT name, average_grade,
    CASE WHEN average_grade >= 4.5 THEN N'Отличник'
         WHEN average_grade >= 3.5 THEN N'Хорошист'
         WHEN average_grade >= 3.0 THEN N'Успевает'
         ELSE N'Есть задолженности' END AS result
FROM college.students;
```

## Задания 6-7

Для цены 1500 скидка составляет 10%, итоговая цена - 1350.

```
DECLARE @price DECIMAL(10,2) = 1500;
DECLARE @discount DECIMAL(5,2);
IF @price >= 1000
    SET @discount = 10;
ELSE
    SET @discount = 5;
SELECT @price AS price, @discount AS discount,
       @price - @price * @discount / 100 AS final_price;
GO

SELECT name, age,
    CASE WHEN age < 18 THEN N'Младше 18'
         WHEN age <= 19 THEN N'18-19 лет'
         ELSE N'20+' END AS age_group
FROM college.students;
```

## Задание 8

Для id = 1 получаем: Иван, 18, 4.70, 18-19 лет, Отличник.

```
DECLARE @studentId INT = 1;
DECLARE @studentName NVARCHAR(100);
DECLARE @age INT;
DECLARE @averageGrade DECIMAL(4,2);

SELECT @studentName = name, @age = age,
       @averageGrade = average_grade
FROM college.students WHERE id = @studentId;

SELECT @studentName AS name, @age AS age,
       @averageGrade AS average_grade,
    CASE WHEN @age < 18 THEN N'Младше 18'
         WHEN @age <= 19 THEN N'18-19 лет'
         ELSE N'20+' END AS age_group,
    CASE WHEN @averageGrade >= 4.5 THEN N'Отличник'
         WHEN @averageGrade >= 3.5 THEN N'Хорошист'
         WHEN @averageGrade >= 3.0 THEN N'Успевает'
         ELSE N'Есть задолженности' END AS result;
```